
Symbolic PDE Conditioning in Fluid Flow Models

Akhil Sadam

CCSE

Massachusetts Institute of Technology

Cambridge, MA 02139

a1744874@mit.edu

Abstract

State-of-the-art fluid models remain limited multimodal capability, restricting their operational use. In this work, we investigate how explicit PDE conditioning and temporal priors generate a simple, interpretable foundation model for fluid dynamics. We propose a hybrid architecture that couples a large language model (LLM)-inspired PDE encoder with a conditional flow-matching model for trajectory sampling. The PDE encoder provides a human-interpretable, symbolic representation of the dynamics, while the flow-matching model enables stable, physically consistent forecasts. We study the interaction between implicit and explicit PDE conditioning, and find the following: implicit conditioning through temporal context can recover nearly equivalent dynamical structure with scale. Eventually, we hope to enable inverse PDE inference from flows and zero-shot forecasting from unseen PDEs, towards equation discovery for realistic fluid flows, such as those in the ocean and atmosphere. Overall, our results highlight a trade-off between symbolic and temporal conditioning in generative physics models, providing insights into the design of multimodal foundation models for scientific simulation.

1 Introduction

1.1 Motivation and related work

A central challenge in atmospheric and oceanic dynamics is how to identify and interpret the governing physical laws (PDEs) of complex fluid flow. For example, in ocean science one may have an approximate PDE that generally describes the flow, but is clearly inaccurate or missing important physics. From measurements, or more accurate simulations, one seeks to identify the true PDE from the flow fields (an multivariate, multimodal inverse problem).

At the same time, fluid simulations are expected to operate across increasingly multimodal data, including simulated flow-fields, human-interpretable PDE laws, and sparse observational measurements. A fundamental question is then how these laws (which are explicit physical priors) interact with implicit structure in data-driven models.

To formalize this multimodal setting, we categorize common input representations in fluid modeling into a minimal set of modalities in Table 1, each decomposed into their multimodal aspects per Liang et al. [2023]. These modalities induce a set of fundamental capabilities (Table 2) that a fluid foundation model must support, spanning interpretation, forecasting, and cross-domain transfer.

With these operational requirements (Table 2), we evaluate existing fluid modeling approaches across representation, forecasting, and transfer capabilities in Table 3.

- *Interpretability and symbolic alignment:* Recent work in physics-informed and symbolic foundation models, such as PI-MFM [Zhu et al., 2025] and PDEformer-1 [Ye et al., 2025]

Table 1: Simplified list of input representations and corresponding modalities

Input	Representation	Modalities	Physical Fidelity
I1	Low-resolution simulation	Complete state (video)	Unresolved and idealized
I2	High-resolution simulation	Complete state (video)	Idealized
I3	Symbolic	PDE (text subset), initial field (image), forcing field (image / video)	Partially-idealized
I4	High-resolution realistic flow	Complete state (video)	Complete
I5	Sparse measurement data	Partial state (masked video)	Incomplete (partial observation)

Table 2: Fundamental capabilities of fluid models

Task	Inputs	Outputs	Involves	Goal
Interpretability	All except I3	I3	Representation Alignment	Recover a symbolic representation
Stable Forecasting	Any	At future time	Reasoning Generation	Conditional video distributions
Resolution Closure	I1, I5	I2	Transference	Correct for unresolved physics
Physics Closure	I2, I5	I4	Transference	Correct for unknown physics
Translation	I1, I2	I4	Transference	Maps idealized $\leftrightarrow$ realistic

incorporate explicit symbolic PDE structure (as text or graphs representations respectively) and align latent representations for prediction. These approaches remain deterministic (do not produce a distribution) and are not capable enough for higher-dimensional chaotic PDEs. Classical neural closure methods such as Generalized Neural Closure [Gupta and Lermusiaux, 2023] avoid explicit representation learning by using a predefined function library, trading expressiveness for interpretability.

- *Stable forecasting and generative rollout:* On the forecasting side, a lot of work has been done towards solver stability, with latest weather-prediction models like FourCastNet3 [Bonev et al., 2025], Thermalizer [Pedersen et al., 2025], using diffusion to stabilize long-term forecast (rollout) while smaller models like Ain and LUCIE [Guan et al., 2025] add spectral reweighting or PDE integrators to enforce stability. However, none of these methods align their latent representations with symbolic PDEs, limiting applicability and general transference (generalization).

Clearly, no existing approach jointly supports interpretability, stable generative forecasting, and cross-representation transfer within a unified framework. Our contributions are as follows:

- **A unified PDE-conditioned generative framework for fluid dynamics.** We introduce a hybrid architecture that combines a large language model (LLM)-based PDE encoder with a conditional flow-matching model for trajectory generation. This design enables joint conditioning on both symbolic PDE descriptions and temporal observations, bridging interpretable physics and generative sequence modeling. Our design builds on Zhu et al. [2025], extending to higher dimension and stochastic generation.
- **Explicit symbolic vs. implicit conditioning.** We analyze the interaction between explicit PDE conditioning and implicit generative priors, and show that implicit conditioning through temporal context can recover nearly equivalent dynamical structure with scale, while explicit PDE conditioning remains causally important.
- **Preliminary results on inverse PDE inference.** We present early experiments on inverse PDE inference from trajectories and eventually zero-shot out-of-distribution generalization, suggesting equation discovery and controllable simulation for complex fluid systems such as oceanic and atmospheric flows. [I’m leaving this in since I did train this part, but results are too poor to discuss..., so they were omitted.]

Table 3: Multimodal aspects of neural fluid models. We shorthand the capabilities as follows: Representation, Alignment, Reasoning, Generation, Transference as Rep, Ali, Rea, Gen, and Tra, respectively. Quantification is not considered as it is relatively straightforward.[Note, remove table for final paper?]

Area	Paper	Contribution	Rep	Ali	Rea	Gen	Tra
Interpretability	PI-MFM Zhu et al. [2025]	Maps symbolic PDEs (Polish notation) to neural operators (zero-shot).	✓	✓			✓
	PDEformer-1 Ye et al. [2025]	PDE encoded as directed acyclic graphs with mesh-free solutions.	✓	✓			✓
	Generalized Neural Closure (2023) Gupta and Lermusiaux [2023]	Interpretable, library-based closures		✓			✓
Stable Forecasting	FourCastNet 3 Bonev et al. [2025]	Latent-space geometric neural operator with diffusion			✓	✓	
	Thermalizer Pedersen et al. [2025]	Stabilized rollout via diffusion			✓	✓	
	LUCIE Guan et al. [2025]	Stabilized rollout by enforcing PDE integrator			✓	✓	
Resolution Closure	Unpaired Downscaling Bischoff and Deck [2024]	Diffusion bridge between coarse and fine fluid simulations.		✓		✓	✓
	Climate in a Bottle Brenowitz et al. [2025]	Cascaded diffusion for kilometer-scale atmospheric generation.		✓	✓	✓	✓
	Guided... and Cond. Turbulence Babu et al. [2025b]	Information transfer from I5→I2 in diffusion models		✓		✓	✓
Physics Closure	Neural Closure Models Gupta and Lermusiaux [2021]	Temporal closures from an adjoint and RNN viewpoint (sequence to image).		✓	✓		✓
	A-posteriori Learning Frezat et al. [2022]	Online closure learning for idealized QG turbulence.		✓	✓		✓
	Adjoint QG (2024) Yan et al. [2024]	Adjoint-based online learning for two-layer turbulence.		✓	✓		✓
	Stochastic Diffusion Closure Dong et al. [2025]	Diffusion & FNO based stochastic closure			✓	✓	✓

2 Methodology

2.1 Datasets

2.1.1 PDE Data

Following Zhu et al. [2025], we encode generated PDEs with reverse Polish notation (RPN) as in the 4 simple examples below. Only the right-hand side of the PDE is generated, with the left-hand side representing $\frac{\partial q}{\partial t}$. Here, ψ , u , v , and jacobian represent the standard quasigeostrophic field variables (vorticity, u-velocity, v-velocity) and the jacobian (advection) operator, respectively.

```

0.37 3 psi + sin q 0 y jacobian - + +
q q u cos + cos + dy
x y dx * dy
x psi * cos

```

2.1.2 Flow Data

Idealized quasigeostrophic (QG) flow is a common simplification used to approximate large scale atmospheric/oceanic flow. We generate data through an adapted variant of Babu et al. [2025a]’s QG

solver, which solves the PDE directly. The procedure is quite simple; for each PDE type among standard QG variations, we randomly generate different parameters and start a run on the MIT ORCD Engaging cluster. Upon termination the vorticity field is extracted as a NumPy file for training.

We generate paired timeseries (one batched timeseries per RPN PDE).

2.2 Architecture and Forward Problem Formulation

2.2.1 Contrastive PDE Encoder

We describe the PDE encoder architecture (1) and objectives.

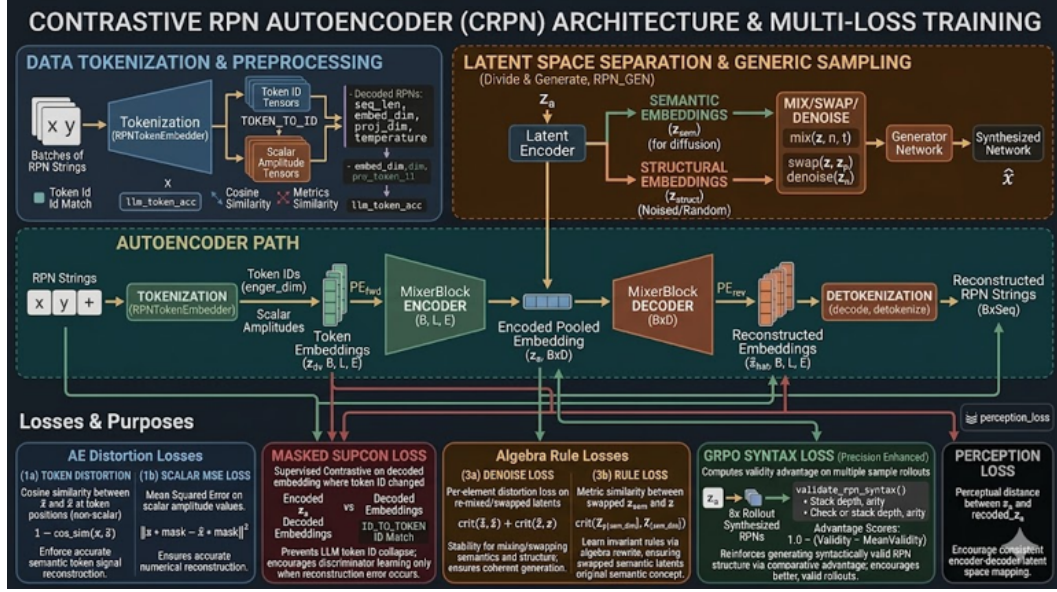


Figure 1: PDE encoder architecture

Let x denote an input expression represented as an RPN token sequence. The model first tokenizes x into token IDs and amplitudes, which are embedded into a continuous sequence representation via a learned token embedder. We denote this embedding as

$$h = \text{Embed}(x) \in \mathbb{R}^{L \times d}. \quad (1)$$

A projection head f_θ , implemented as part of an RPN autoencoder, maps the embedded sequence into a pooled latent representation:

$$z = f_\theta(h, x) \in \mathbb{R}^d, \quad (2)$$

where padding tokens are masked out during pooling.

Masked reconstruction objective. The decoder reconstructs embeddings $\hat{h}$ from z , producing both token-level and scalar-sensitive errors. We use a normalized reconstruction loss, and to account for structured token positions, we apply a masked geometric loss where cosine distance is computed only on non-padding tokens and normalized reconstruction is computed on the scalar slots.

$$\mathcal{L}_{\text{rel}} = \frac{\|\hat{h} - h\|_2^2}{\text{Var}(h) + \epsilon}. \quad (3)$$

$$\mathcal{L}_{\text{mask}} = \mathcal{L}_{\text{cos}}^{\text{token}} + \mathcal{L}_{\text{mse}}^{\text{scalar}}, \quad (4)$$

Perception (consistency) loss We decode and re-encode the reconstruction to enforce latent stability, and minimize a perception consistency loss.

$$\tilde{z} = f_\theta(\text{Dec}(z)), \quad (5)$$

$$\mathcal{L}_{\text{perc}} = \|\tilde{z} - z\|_2^2. \quad (6)$$

Masked supervised contrastive loss To prevent representational collapse and encourage token discriminability, we apply a masked supervised contrastive loss [?] only on positions where reconstruction changes token identity. Let i, j index active token positions:

$$\mathcal{L}_{\text{supcon}} = -\frac{1}{|A|} \sum_{i \in A} \frac{1}{|P(i)|} \sum_{j \in P(i)} \log \frac{\exp(z_i^\top z_j / \tau)}{\sum_{k \neq i} \exp(z_i^\top z_k / \tau)}, \quad (7)$$

where A is the set of altered tokens and $P(i)$ are positives sharing the same original token ID.

Algebraic invariance via latent mixing and swap perturbations. We sample algebraically equivalent rewrites $x^+ \sim \mathcal{T}(x)$ and encode both inputs into a latent space:

$$z = f_\theta(x), \quad z^+ = f_\theta(x^+), \quad (8)$$

where the latent is decomposed into semantic and structural components. We inject Gaussian noise into the structural subspace while preserving semantic content:

$$\tilde{z} = tz + (1-t)n, \quad n \sim \mathcal{N}(0, 1), \quad t \sim \mathcal{U}(0, 1). \quad (9)$$

To enforce algebraic invariance, we swap semantic components between paired latents:

$$\tilde{z}, \tilde{z}^+ \leftarrow \text{Swap}_{\text{sem}}(\tilde{z}, \tilde{z}^+), \quad (10)$$

where only the first semantic coordinates are exchanged. A generator $\mathcal{G}_{\text{LM}}$ then performs denoising in latent space followed by projection back to representation space:

$$\hat{x} = \mathcal{G}_{\text{LM}}(\tilde{z}), \quad \hat{x}^+ = \mathcal{G}_{\text{LM}}(\tilde{z}^+). \quad (11)$$

We enforce reconstruction consistency at both representation and latent levels:

$$\mathcal{L}_{\text{denoise}} = \mathcal{L}(\hat{x}, x) + \mathcal{L}(\hat{x}^+, x^+) + \mathcal{L}(f_\theta(\hat{x}), z) + \mathcal{L}(f_\theta(\hat{x}^+), z^+). \quad (12)$$

Finally, we explicitly regularize semantic invariance between paired encodings:

$$\mathcal{L}_{\text{semantic}} = \|z_{\text{sem}} - z_{\text{sem}}^+\|_2^2, \quad (13)$$

The generator $\mathcal{G}_{\text{LM}}$ is thus trained implicitly through this denoising and swap-consistency objective rather than a standalone contrastive loss.

Final objective. The full training objective is:

$$\mathcal{L} = \mathcal{L}_{\text{mask}} + \mathcal{L}_{\text{rel}} + \mathcal{L}_{\text{perc}} + \mathcal{L}_{\text{supcon}} + \mathcal{L}_{\text{semantic}} + \mathcal{L}_{\text{denoise}} \quad (14)$$

We ablate the semantic and denoise (the algebraic invariance) terms in our experiments.

2.2.2 Conditional Flow-Matching Operator Model

Given an initial condition $x_0 \in \mathbb{R}^{C \times H \times W}$ and a symbolic latent code $z \in \mathbb{R}^d$ from the Contrastive PDE Encoder, we model PDE dynamics via a conditional flow matching formulation.

A symbolic RPN input is encoded as $z = f_\theta(x_{\text{RPN}})$ which parameterizes the underlying dynamics. The latent z is injected into the generator at every step via feature-wise linear modulation (FiLM), enabling global control of the evolution.

Flow Matching Dynamics. We model time evolution as a continuous transformation between a source distribution (noise) and the target data distribution. Given a clean state x_t and noise $\epsilon \sim \mathcal{N}(0, I)$, we define an interpolation $x(\alpha)$.

$$x(\alpha) = \alpha x_t + (1 - \alpha)\epsilon, \quad \alpha \sim \mathcal{U}(0, 1). \quad (15)$$

The model is formulated as x-prediction, but trained with an equivalent relative v-prediction loss such that v_ϕ defines a straight-line flow between noise and data Li and He [2026].

$$v_\phi(x(\alpha), \alpha; x_0, z) \approx x_t - \epsilon, \quad (16)$$

The generator g_ϕ implements a single-step update by integrating the learned flow. Starting from x_0 , predictions are generated recursively, corresponding to repeatedly solving a learned conditional flow, where the latent z fixes the governing PDE.

$$\hat{x}_{t+1} = g_\phi(\hat{x}_t, x_0; z), \quad \hat{x}_0 = x_0. \quad (17)$$

As the formulation we have thus developed generates $\hat{x}_{t+1}$ given z_{PDE} , we denote this the forward problem, as opposed to the inverse problem of identifying z_{PDE} from x_{flow} .

2.3 Flow-matching Model

As most pretrained generative models are quite expensive (or do not fit in memory, such as Flux, PixArt and more), we use a custom ViT-style latent patch diffusion.

2.3.1 Fast Unconditional Generation

Due to the relative isotropy of quasigeostrophic turbulence, our ViT-style latent patch diffusion is quite efficient. We can train on 512x512 images of a single PDE and generate the following 2048x2048 image (2) at about 100ms for 25 steps with a second-order Adams-Bashforth-trapezoidal scheme. After conditioning, generation does become much slower (not optimized yet), but we still

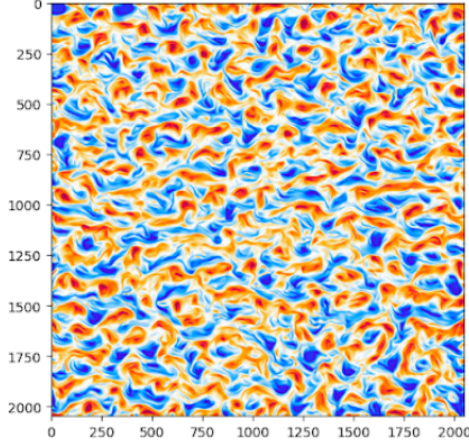


Figure 2: Unconditional FMM generation in 100-130ms per image

get fairly accurate results for a single PDE dataset (3). Clearly, the dynamics signal $\hat{x}_{t+1} - x_t$ is much larger than the error $\hat{x}_{t+1} - x_{t+1}$, so the model is predicting well. Note that here we predict a single step of about 800x the numerical timestep, and thus achieve comparable speed to traditional, ultrafast pseudospectral solvers.

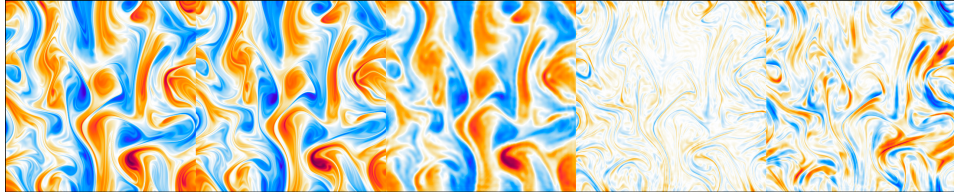


Figure 3: From left to right (rotated images): $x_t, x_{t+1}, \hat{x}_{t+1}, \hat{x}_{t+1} - x_{t+1}, \hat{x}_{t+1} - x_t$

3 Results and Conclusions

3.1 Contrastive PDE Encoder

First, we benchmark our PDE encoder against a LoRA-finetuned QWEN 2.5 model (see Appendix, Fig. 5 for details). In Table 4, We find that the QWEN model hallucinates natural language tokens and overfits strongly, while the PDE encoder performs well. Due to the small dataset size, the PDE encoder can reproduce the PDEs well but not exactly. Numerical values are distorted and sampling through the generator $\mathcal{G}_{LM}$ does not properly learn rewrites. Without the algebra losses, the generator is not sufficiently trained, and outputs mostly random tokens.

Model	Token Acc	Num. RMSE	Parameters	Trainable Parameters
PDE encoder with algebra losses	0.905	0.073	1.3M	1.3M
PDE encoder w/o algebra	0.905	0.096	1.3M	1.3M
QWEN LoRA-finetuned LLM	0.048	N/A	1.5B	24.5M

Table 4: Performance comparison across PDE models on token accuracy (weighted accuracy of token prediction) and numerical RMSE of the coefficients.

Input strings

0.37 3 psi + sin q 0 y jacobian - + +
q q u cos + cos + dy

PDE encoder

0.314546 3.034548 psi + sin q y y jacobian - + +
q q u cos + cos + dy

PDE encoder, sampled rewrite

0.275928 2.961881 psi + sin q 0.091960 y v - + +
q q u cos + cos + dy

PDE encoder w/o algebra, sampled rewrite

cos 0.290347 v invlap jacobian jacobian dx psi cos ...

3.2 Flow-Matching Performance

Now that we have a good PDE encoder, we ablate the flow-matching performance across PDE conditioning, which we notate as VLM (vision and language) or VM (vision only). We define two metrics, RelMSE or relative MSE, and PMSE, or a percentage score that evaluates the correctness of the prediction. A PMSE of 100% corresponds to a completely useless prediction. These metrics are averaged across 16 different testing PDEs, and 40 different prediction times along the trajectory to provide a comprehensive score.

$$\text{RelMSE} = \frac{\mathbb{E} [\|\hat{x}_t - x_t\|_2^2]}{\mathbb{E} [\|x_t\|_2^2]} \quad \text{or relative MSE} \quad (18)$$

$$\text{PMSE} = \frac{\mathbb{E} [\|(\hat{x}_t - x_t)\|_2^2]}{\mathbb{E} [\|(\hat{x}_t - x_{t-1})\|_2^2]} \quad (19)$$

We find that the baseline (VM) model without the PDE encoder performs quite well in nearly all scenarios, performing at or better than the (VLM) combined model (Table 5). As we decrease the layers and distill capacity, the combined model starts to reach and possibly beat the baseline model, implying that the implicit PDE conditioning from the context frame requires sufficient network capacity, and below this level, the model must rely on explicit PDE conditioning (though further tests at even smaller architectures are needed to confirm).

Notice that the combined (VLM) model does not ignore the PDE conditioning; on the contrary, it is highly dependent on the conditioning. Permuting the PDE information by batch (so it receives incorrect PDE conditioning) results in much larger error). One way to explain this behavior is via edge case; at minimum capacity, we have a standard PDE solver which solely uses explicit PDE conditioning, while at maximum capacity, we expect the network to be able to perform complete zero-shot generalization. Also note that the algebra losses do not matter to the flow generation; since they only separate out syntactical information for PDE (text) generation.

Model	Epochs	Layers	RelMSE	PMSE (Err / Signal)
VLM	240	5	0.038	11.2%
VLM w/o algebra	240	5	0.04	11.3%
VM (no PDE cond.)	240	5	0.030	8.3%
VLM (incorrect PDE cond.)	240	5	0.149	48.0%
VLM w/o algebra (incorrect PDE cond.)	240	5	0.153	47.6%
VLM	400	5	0.034	9.1%
VM (no PDE cond.)	400	5	0.032	8.6%
VLM (incorrect PDE cond.)	400	5	0.118	35.8%
VLM-sm	(+50)	2	0.031	8.4%
VM-sm (no PDE cond.)	(+50)	2	0.032	8.2%
VLM-sm (incorrect PDE cond.)	(+50)	2	0.102	31.1%

Table 5: Flow-matching performance across PDE-conditioned models. Lower is better for both metrics. The small models are essentially distilled variants that were trained from 50 additional epochs to a total of 450 epochs.

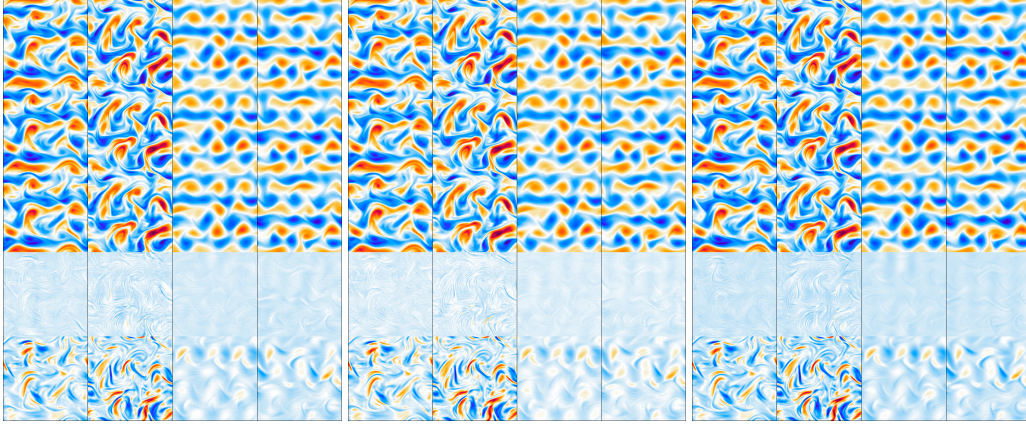


Figure 4: From left to right: VLM, VLM w/o algebra, VM for 4 different PDEs (each column) at a single time (trained for 240 epochs).

From top to bottom: x_{t-1} , the context frame, x_t , the target frame after 800 numerical timesteps, $\hat{x}_t$, the model prediction, $\hat{x}_t - x_t$, the model error, and $\hat{x}_t - x_{t-1}$, the model signal.

What’s most interesting is that at scale, the baseline (VM) model outperforms the combined (VLM) model, implying that the PDE encoder either provides noisy or hard-to-understand information, more so than the information from a single context image.

On the other hand, if we want to identify the PDE from the flow, as in oceanic/atmospheric tasks, the explicit conditioning of the VLM allows interpretability. In particular, we can optimize the latent PDE representation such that the VLM produces the right output, and then use the PDE autoencoder to decode for a PDE string, that is human-interpretable. We tried this briefly, but the autoencoding is too multi-valued to produced a clean PDE at present.

So there is a tradeoff between the VLM and VM for interpretability vs performance.

We show a couple example PDEs and model predictions in Fig. 4.

3.3 Future Work

- Clearly, the PDE model currently produces too noisy of a latent. One direction is to improve the PDE model, and understand what is a noisy vs clean latent.
- Another direction we have been working on is the inverse optimization, to find the latent PDE representation from the flow. This somewhat works, in the sense that we can find a latent representation that produces the correct flow field, but the latent representation does not decode well. To this end, we would need to somehow regularize the PDE latent space.

- Finally, one natural task that we did not test includes out-of-distribution PDEs. Since we can consider the VLM to extend Zhu et al. [2025] to 2D and diffusion models, we can also do out-of-distribution PDEs. We did not test this due to a lack of training data (which meant the model would not learn compositionality). One avenue for future work would involve adding self-supervised losses (train directly against the PDE solver) to alleviate the training data issue (though we would still require a prior distribution to start the PDE-solver from).
- Lastly, one simple task that would be required for application is rollout. So far all the metrics are single-step prediction; to do forecasts we need to do rollout. We have tested this briefly, and the accuracy is low, but still stable unlike other methods as we use a diffusion model approach.

A QWEN Baseline Architecture

We repeat most of the losses as found in 1, but remove the additional tokenization losses, and use a standard teacher-forced, next-token prediction framework (5). The same generator was used in both models for consistency.

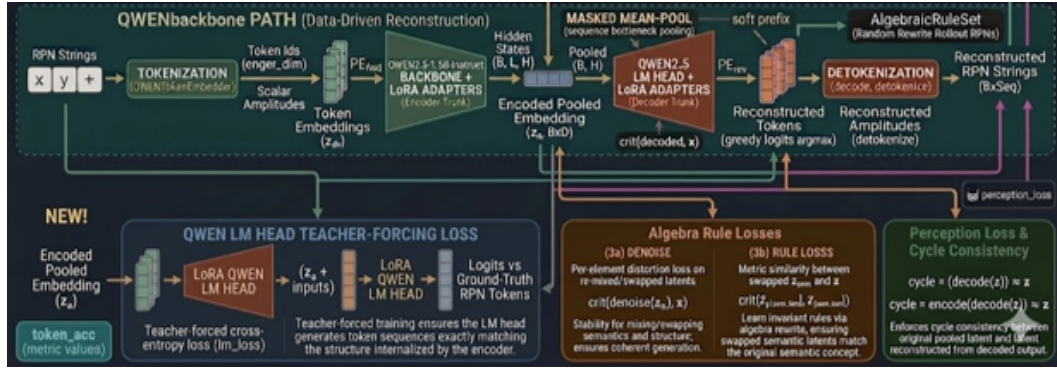


Figure 5:

B Code and Reproducibility

The *autoencoders* repository has the main code, with additional code for simulation and PDE encoding in the *qg* repository. Finally, additional notes are included in the *MMAI* repository; this work would not have happened without the kind assistance of Dr.Liang, the *MMAI* TAs, and the course!

ChatGPT was used to help write algorithmic portions to save time. All LLM-generated text was carefully validated by the authors.

- <https://github.com/akhilsadam/autoencoders>
- <https://github.com/akhilsadam/qg>
- <https://github.com/akhilsadam/MMAI/tree/master>

Acknowledgments and Disclosure of Funding

References

- Ainslie, C. (AM) - A Theoretical Eigenanalysis Framework for Neural Autoregressive Models of Multiscale Chaotic Dynamics. URL <https://calendar.ucsc.edu/event/ainslie-c-am-a-theoretical-eigenanalysis-framework-for-neural-autoregressive-models-of-multiscale-chaotic-dynamics>
- Anantha Narayanan Suresh Babu, Akhil Sadam, and Pierre F. J. Lermusiaux. Evaluation of Analytical Turbulence Closures for Quasi-Geostrophic Ocean Flows with Coastal Boundaries. In *OCEANS 2025 - Great Lakes*,

- pages 01–10, September 2025a. doi: 10.23919/OCEANS59106.2025.11245082. URL <http://arxiv.org/abs/2508.06678>. arXiv:2508.06678 [physics].
- Anantha Narayanan Suresh Babu, Akhil Sadam, and Pierre F. J. Lermusiaux. Guided Unconditional and Conditional Generative Models for Super-Resolution and Inference of Quasi-Geostrophic Turbulence, November 2025b. URL <http://arxiv.org/abs/2507.00719>. arXiv:2507.00719 [physics].
- Tobias Bischoff and Katherine Deck. Unpaired Downscaling of Fluid Flows with Diffusion Bridges. May 2024. doi: 10.1175/AIES-D-23-0039.1. URL <https://journals.ametsoc.org/view/journals/aies/3/2/AIES-D-23-0039.1.xml>.
- Boris Bonev, Thorsten Kurth, Ankur Mahesh, Mauro Bisson, Jean Kossaifi, Karthik Kashinath, Anima Anandkumar, William D. Collins, Michael S. Pritchard, and Alexander Keller. FourCastNet 3: A geometric approach to probabilistic machine-learning weather forecasting at scale, July 2025. URL <http://arxiv.org/abs/2507.12144>. arXiv:2507.12144 [cs].
- Noah D. Brenowitz, Tao Ge, Akshay Subramaniam, Peter Manshausen, Aayush Gupta, David M. Hall, Morteza Mardani, Arash Vahdat, Karthik Kashinath, and Michael S. Pritchard. Climate in a Bottle: Towards a Generative Foundation Model for the Kilometer-Scale Global Atmosphere, July 2025. URL <http://arxiv.org/abs/2505.06474>. arXiv:2505.06474 [physics].
- Xinghao Dong, Chuanqi Chen, and Jin-Long Wu. Data-Driven Stochastic Closure Modeling via Conditional Diffusion Model and Neural Operator, March 2025. URL <http://arxiv.org/abs/2408.02965>. arXiv:2408.02965 [cs].
- Hugo Frezat, Julien Le Sommer, Ronan Fablet, Guillaume Balarac, and Redouane Lguensat. A Posteriori Learning for Quasi-Geostrophic Turbulence Parametrization. *Journal of Advances in Modeling Earth Systems*, 14(11):e2022MS003124, 2022. ISSN 1942-2466. doi: 10.1029/2022MS003124. URL <https://onlinelibrary.wiley.com/doi/abs/10.1029/2022MS003124>. Number: 11 _eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1029/2022MS003124>.
- Haiwen Guan, Troy Arcomano, Ashesh Chattopadhyay, and Romit Maulik. LUCIE: A Lightweight Uncoupled Climate Emulator with long-term stability and physical consistency for O(1000)-member ensembles, April 2025. URL <http://arxiv.org/abs/2405.16297>. arXiv:2405.16297 [cs].
- Abhinav Gupta and Pierre F. J. Lermusiaux. Neural Closure Models for Dynamical Systems, July 2021. URL <http://arxiv.org/abs/2012.13869>. arXiv:2012.13869.
- Abhinav Gupta and Pierre F. J. Lermusiaux. Generalized Neural Closure Models with Interpretability, May 2023. URL <http://arxiv.org/abs/2301.06198>. arXiv:2301.06198.
- Tianhong Li and Kaiming He. Back to Basics: Let Denoising Generative Models Denoise, January 2026. URL <http://arxiv.org/abs/2511.13720>. arXiv:2511.13720 [cs].
- Paul Pu Liang, Amir Zadeh, and Louis-Philippe Morency. Foundations and Trends in Multimodal Machine Learning: Principles, Challenges, and Open Questions, February 2023. URL <http://arxiv.org/abs/2209.03430>. arXiv:2209.03430 [cs].
- Chris Pedersen, Laure Zanna, and Joan Bruna. Thermalizer: Stable autoregressive neural emulation of spatiotemporal chaos, March 2025. URL <http://arxiv.org/abs/2503.18731>. arXiv:2503.18731 [cs].
- Fei Er Yan, Hugo Frezat, Julien Le Sommer, Julian Mak, and Karl Otness. Adjoint-based online learning of two-layer quasi-geostrophic baroclinic turbulence, November 2024. URL <http://arxiv.org/abs/2411.14106>. Issue: arXiv:2411.14106 arXiv:2411.14106 [physics].
- Zhanhong Ye, Xiang Huang, Leheng Chen, Zining Liu, Bingyang Wu, Hongsheng Liu, Zidong Wang, and Bin Dong. PDEformer-1: A Foundation Model for One-Dimensional Partial Differential Equations, January 2025. URL <http://arxiv.org/abs/2407.06664>. arXiv:2407.06664 [math].
- Min Zhu, Jingmin Sun, Zecheng Zhang, Hayden Schaeffer, and Lu Lu. PI-MFM: Physics-informed multimodal foundation model for solving partial differential equations, December 2025. URL <http://arxiv.org/abs/2512.23056>. arXiv:2512.23056 [cs].